

AI Agent Canvas

Complete Reference Guide

Build intelligent AI agents with .NET 9 and Microsoft Agent Framework.
From a single standalone agent to a coordinated multi-agent ecosystem.

Table of Contents

Guide

1. Introduction
2. AI Agents
3. Features
4. Libraries
5. Building an Agent
6. Context Domains
7. MCP and RAG
8. Multi-Agent Setup
9. Architecture
10. Behavior Patterns

Appendix

User Guide

Reference

AG-UI Protocol
Security and Governance
Platform Internals

1. What Is AI Agent Canvas

AI Agent Canvas is an agent development platform built on .NET 9 and Microsoft Agent Framework. It provides the runtime, orchestration layer, and pre-built capabilities to build AI agents -- from a single standalone agent to a coordinated multi-agent ecosystem.

The platform sits between your business logic and the LLM. You write the pieces that make your agent unique (persona, tools, domain knowledge), and the platform handles the rest: the agent execution loop, context injection, tool governance, streaming, state management, and multi-agent coordination.

Scales of Deployment

Scale	Description
Standalone agent	One agent, one process, one domain focus. The simplest deployment: a single agent with its own persona, tools, and context, running in one host. Good for task-specific use cases like a compliance checker or document analyst.
Multi-agent (in-process)	Multiple specialists in one process, coordinated through handoff and background delegation. Agents share a host and can transfer conversations between each other or spin off parallel work. The user talks to one agent at a time, but the system routes to whichever specialist fits.
Agent ecosystem	Multiple hosts connected via A2A protocol over HTTP, with independent deployment and security boundaries. Each host runs its own agents and exposes them as services. Agents discover each other through AgentCards and communicate across network boundaries, owned and deployed by different teams.

The same codebase supports all three. You start with one agent and add more as the problem demands it. A standalone agent can become part of a multi-agent system by registering it in the agent registry and defining handoff rules. That multi-agent system can later federate with other hosts over A2A without changing the agent code.

What the Platform Provides

The platform handles the infrastructure so you can focus on domain logic:

- **Agent execution loop** -- the reason-act-observe cycle that drives every agent interaction
- **Context injection** -- automatic assembly of persona, entity memory, user profile, guardrails, and domain knowledge into the system prompt
- **Tool governance** -- policy-based filtering and approval rules applied before any tool executes

- **Streaming** -- real-time Server-Sent Events via the AG-UI protocol, delivering text, tool calls, and state updates to the frontend as they happen
- **State management** -- persistent chat history, entity memory, and scheduled task storage backed by SQLite
- **Multi-agent coordination** -- handoff, background delegation, async messaging, and workflow orchestration across agents

What You Provide

Your job is to define what makes each agent distinct:

- **Persona** -- the agent's identity, tone, expertise, and behavioral instructions
- **Tools** -- the functions the agent can call to take action in your domain
- **Domain knowledge** -- RAG sources, entity definitions, and contextual data the agent needs to reason well
- **Guardrails** -- boundaries on what the agent should and should not do
- **Skills and workflows** -- multi-step procedures the agent follows to complete complex tasks

2. What Is an AI Agent

An AI agent is software that uses an LLM to reason about a goal, decide what actions to take, execute them through tools, and iterate until the goal is met. Unlike a chatbot that generates text in response to prompts, an agent acts on your behalf -- it reads data, calls APIs, modifies systems, and makes decisions across multiple steps.

The Agent Loop

Every agent interaction follows the same cycle:

```
User message -> Context injection (persona, memory, guardrails, domain knowledge) -> LLM reasoning (understand the goal, plan next steps) -> Tool calls (execute actions via functions, APIs, or external systems) -> Tool results (observe outcomes) -> LLM reasoning (evaluate results, decide if done or continue) -> Response (deliver the final answer or ask for clarification)
```

This loop repeats as many times as needed. A simple question might take one pass. A complex task -- researching a topic, pulling data from three systems, formatting a report -- might take ten. The agent decides when it has enough information to respond.

In code, this loop is managed by the agent runtime. Here is a simplified version of what happens on each turn:

```
// The runtime assembles context and runs the loop var systemPrompt = contextManager.BuildSystemPrompt(agent, user, conversation); var messages = new List<ChatMessage> { new SystemChatMessage(systemPrompt) }; messages.AddRange(conversation.History); messages.Add(new UserChatMessage(userInput)); // The FunctionInvokingChatClient handles the tool-call loop automatically var response = await chatClient.GetResponseAsync(messages, options);
```

The `FunctionInvokingChatClient` from `Microsoft.Extensions.AI` intercepts tool-call responses from the LLM, executes the corresponding functions, feeds results back, and lets the LLM continue reasoning -- all within a single call.

What Makes Agents Different from LLM Wrappers

A thin LLM wrapper sends a prompt and returns the response. An agent goes further:

- **Tool use** -- agents call functions to interact with external systems. They don't just describe what should happen; they do it. A wrapper tells you the API call to make. An agent makes the call.
- **Context awareness** -- agents maintain state across turns: who the user is, what entities are in play, what happened earlier in the conversation, and what domain knowledge applies. Each turn builds on the last.
- **Multi-step reasoning** -- agents break complex goals into steps, execute them in sequence, and adapt based on intermediate results. If step 3 fails, the agent reasons about why and tries a different approach.
- **Persistence** -- agents remember across sessions. Entity memory, user preferences, and conversation history survive restarts. The agent picks up where it left off.

Use Cases

Domain	What the Agent Does
Financial services	Monitors portfolio risk, flags compliance violations, and generates regulatory reports from transaction data.
IT operations	Triages incidents from monitoring alerts, correlates logs across systems, and executes runbook steps to resolve issues.
Customer support	Resolves tickets by looking up account data, applying business rules, and taking actions like issuing refunds or updating records.
Legal and compliance	Reviews contracts against policy checklists, extracts key clauses, and flags deviations for human review.
Healthcare	Summarizes patient records, cross-references symptoms with clinical guidelines, and prepares pre-visit briefs for clinicians.
Software engineering	Reviews pull requests, generates test cases from specifications, and investigates build failures by reading logs and code.
E-commerce	Manages product catalog updates, processes return requests end-to-end, and generates pricing recommendations from sales data.

3. Features

Agent Capabilities

These features apply to every agent, whether running standalone or as part of a multi-agent system.

Feature	Description
Dynamic personas	Each agent has a persona that defines its identity, expertise, tone, and behavioral rules. Personas are injected into the system prompt at runtime and can be swapped or layered without changing code.
Persistent context	Domain-specific context (facts, rules, reference material) is loaded into the agent's system prompt on every turn. Context is defined in markdown files and assembled by the context manager.
Entity memory	Agents remember key entities (people, projects, systems, accounts) across conversations. Entities are stored in SQLite and recalled when relevant, giving the agent long-term awareness of the domain it operates in.
Guardrails	Behavioral boundaries defined in markdown that constrain what the agent will and will not do. Guardrails are injected into the system prompt alongside the persona, enforcing policy at the reasoning level.
User profiles	The agent knows who it is talking to -- name, role, preferences, and permissions. User profiles are loaded at the start of each session and shape how the agent responds.
Skills	Named, multi-step procedures the agent can execute. Skills are registered as tools and contain structured instructions the agent follows. They turn complex workflows into repeatable, reliable operations.
Workflows	Orchestrated sequences of steps involving multiple tools, decisions, and checkpoints. Workflows can be defined in code or declared in YAML for sequential, concurrent, or conditional execution.
Scheduling	Agents can schedule tasks for future or recurring execution. Scheduled tasks are persisted in SQLite and executed by a background service, enabling agents to set reminders, run periodic checks, or defer work.

Feature	Description
MCP connections	Agents connect to external data sources and tools through the Model Context Protocol. MCP servers are configured at runtime via <code>McpConnectionManager</code> and their tools appear alongside native agent tools.
RAG pipeline	Retrieval-augmented generation backed by a SQLite vector store. Documents are chunked, embedded, and stored locally. At query time, the agent retrieves relevant chunks using cosine similarity with FTS5 hybrid search.
System tools	Built-in tools available to every agent: web search, URL reading, file operations, and system utilities. These are registered automatically and subject to the same governance as custom tools.
File workspace	<code>FileAccessProvider</code> gives agents a sandboxed file system for reading, writing, and managing files. Agents can work with uploaded documents, generate outputs, and organize artifacts in their workspace.
Tool governance	Policy-based filtering applied before any tool executes. Governance rules can deny, allow, or require approval for specific tools or tool categories, using a deny-overrides evaluation model.
Tool approval	Interactive approval flow for sensitive tool calls. Users can approve once, approve for the session ("don't ask again"), or deny. Approval decisions are cached so repeated calls to the same tool don't interrupt the flow.
OpenTelemetry	Distributed tracing and metrics for agent operations. Every LLM call, tool invocation, and agent turn is instrumented, providing observability into what the agent did and how long it took.
Real-time streaming	Server-Sent Events via the AG-UI protocol deliver text tokens, tool calls, reasoning steps, and state updates to the frontend as they happen. The user sees the agent thinking and acting in real time.

Multi-Agent Features

These features enable coordination between multiple agents within a single host or across hosts.

Feature	Description
Agent registry	A central registry of all agents in the system, with their personas, tools, and capabilities. Agents discover each other through the registry, which the orchestrator uses to route requests to the right specialist.
Handoff	The <code>handoff_to_agent</code> mechanism transfers an active conversation from one agent to another. The receiving agent gets the conversation context and continues seamlessly. Used when a request falls outside the current agent's expertise.
Background agents	Parallel delegation to agents that work independently in the background. The primary agent continues its conversation while background agents execute tasks concurrently and report results when done.
Agent messaging	Asynchronous mailbox-based communication between agents. Agents can send and receive messages without blocking, enabling coordination patterns that don't require real-time handoff.
A2A protocol	Agent-to-Agent communication over HTTP/JSON across host boundaries. Remote agents are discovered through AgentCards that describe their capabilities. Once connected, remote agents behave like local agents -- the calling agent doesn't need to know whether its peer is in-process or across the network.
Workflow orchestration	Multi-agent workflows that coordinate work across agents in sequential, concurrent, or declarative patterns. Workflows can be defined in code or in YAML, specifying which agents handle which steps, how data flows between them, and what happens on failure.

4. Libraries

AI Agent Canvas is built on a stack of libraries that each handle a specific concern. Understanding what each one does and where it fits will help you navigate the codebase and make informed decisions about what to extend.

Microsoft Agent Framework (MAF)

The core agent runtime. MAF provides `HarnessAgent`, the base class for all agents, along with the middleware pipeline that processes each turn -- context injection, tool resolution, governance checks, and response handling. It defines how agents are registered, how they receive messages, and how multi-agent coordination works (handoff, background delegation, messaging).

Key packages:

- `Microsoft.Agents.AI` -- agent abstractions and the AI processing pipeline
- `Microsoft.Agents.Harness` -- `HarnessAgent`, middleware, and the agent execution loop
- `Microsoft.Agents.Workflows` -- workflow orchestration (sequential, concurrent)
- `Microsoft.Agents.Workflows.Declarative` -- YAML-based workflow definitions

Microsoft.Extensions.AI (MEAI)

The abstraction layer between your agent code and the LLM provider. MEAI defines `IChatClient` for chat completions, `AIFunction` and `AITool` for tool definitions, and `IEmbeddingGenerator` for embeddings. The `FunctionInvokingChatClient` wraps any `IChatClient` and automatically handles the tool-call loop -- intercepting tool-call responses, executing functions, and feeding results back to the model. `DelegatingChatClient` enables middleware patterns like logging, caching, and governance wrapping.

MEAI makes the LLM backend swappable. The platform defaults to Azure AI Foundry, but any provider that implements `IChatClient` works without changing agent code.

Azure AI Foundry (Azure OpenAI)

The default LLM provider. `AzureAIFoundryClientFactory` creates `IChatClient` and `IEmbeddingGenerator` instances configured for Azure OpenAI endpoints. It supports both API key and managed identity authentication, selected through configuration. The factory is registered in DI and consumed by the agent runtime -- agents never talk to Azure directly.

Azure AI Foundry is the default, not a requirement. Swapping to another provider means replacing the client factory registration. The rest of the platform is unaffected.

AG-UI Protocol

The streaming protocol between the agent backend and the frontend. AG-UI uses Server-Sent Events (SSE) to deliver a typed event stream: text deltas, tool call starts and completions, reasoning content, state snapshot and

delta updates, interrupt requests, and run lifecycle events (started, completed, error). The frontend consumes these events to render the agent's output progressively -- the user sees text appearing, tool calls executing, and state changing in real time.

The protocol is defined by a set of event types with structured payloads. The backend emits events as the agent processes each turn; the frontend subscribes and renders.

A2A Protocol (Agent-to-Agent)

Cross-host agent communication over HTTP/JSON. A2A lets agents running in different processes or on different machines collaborate as if they were local. Each host publishes an AgentCard -- a JSON document describing the agent's name, capabilities, and endpoint. Remote agents are discovered by fetching their AgentCard and registered in the local agent registry.

Once registered, a remote agent is called through the same handoff and messaging interfaces as a local agent. The calling agent does not need to know whether its peer is in-process or across the network. A2A handles serialization, transport, and error propagation.

Model Context Protocol (MCP)

A standard protocol for connecting agents to external data sources and tools. MCP servers expose tools (functions the agent can call) and resources (data the agent can read) over a defined interface. The platform's `McpConnectionManager` handles runtime connection lifecycle -- starting, stopping, and reconnecting to MCP servers as configured.

Tools from MCP servers are merged into the agent's tool set and subject to the same governance rules as native tools. This means policy-based filtering, approval flows, and audit logging apply uniformly, regardless of whether a tool is defined in code or provided by an MCP server.

Microsoft Agent Governance Toolkit

Security and compliance middleware for agent operations. The governance toolkit provides two main capabilities: prompt injection detection (scanning inputs for attempts to manipulate the agent's behavior) and policy-based tool filtering using a deny-overrides evaluation model (if any policy denies a tool call, the call is blocked, regardless of other policies that allow it).

Governance decisions generate audit events that record what was evaluated, what the outcome was, and why. These events feed into the platform's observability pipeline for compliance reporting and incident investigation.

Microsoft Purview

Compliance middleware for content classification and data loss prevention (DLP). Purview integration adds a middleware layer that classifies content flowing through the agent -- both inputs and outputs -- against sensitivity labels and DLP policies. If content matches a restricted classification, the middleware can block, redact, or flag it before it reaches the user or an external system.

This is relevant for agents operating in regulated environments where data handling must conform to organizational or regulatory policy.

SQLite + Microsoft.Extensions.VectorData

Local persistence for all agent state. SQLite stores chat history (messages and metadata), entity memory (named entities the agent tracks across sessions), and scheduled tasks (deferred and recurring work). The vector store, built on `Microsoft.Extensions.VectorData`, adds embedding-based retrieval for the RAG pipeline.

The RAG implementation uses a hybrid search strategy: cosine similarity over embeddings for semantic relevance, combined with FTS5 full-text search for keyword precision. Documents are chunked, embedded via `IEmbeddingGenerator`, and stored in SQLite. At query time, both search paths run and results are merged and ranked.

SQLite was chosen for simplicity and portability -- no external database dependency, single-file storage, and good-enough performance for the agent's access patterns.

5. Building an Agent

An agent in AI Agent Canvas is a thin set of seed classes that declare what the agent *is* and what it *knows*. You write six pieces of custom code. The platform handles everything else: runtime execution, streaming, context injection, tool routing, governance, telemetry, and more.

What You Implement

1. A Persona Seed

The persona defines the agent's identity, role, and behavioral instructions. It implements `IPersonaSeed` using the `PersonaSeed` class, which takes a name, description, and freeform instructions string.

```
services.AddSingleton<IPersonaSeed>(new PersonaSeed( name: "financial-analyst", description:
"A financial analyst that uses market data tools to research stocks and companies",
instructions: "" You are a financial analyst assistant. You have access to these tools: -
stock_quote: Get current price and daily change for a ticker from Yahoo Finance -
stock_history: Get historical price data with configurable range (5d, 1mo, 3mo, 6mo, 1y, 5y) -
edgar_company_facts: Fetch SEC EDGAR financial data including revenue, net income, assets, and
EPS When analyzing a stock: 1. Start with stock_quote for the current price and daily movement
2. Use stock_history to show recent trends 3. Use edgar_company_facts for fundamental data
(revenue, earnings, balance sheet) 4. Synthesize the data into a clear summary with specific
numbers Always cite the data source (Yahoo Finance or SEC EDGAR) and note that this is
informational, not investment advice. ""));
```

The interface is minimal:

```
public interface IPersonaSeed { string Name { get; } string Description { get; } string
Instructions { get; } }
```

On first startup, the persona is saved to agent-data/orchestrator/agent/personas/financial-analyst.md. If that file already exists, the seed is skipped -- manual edits are preserved.

2. Tool Providers

Tools are registered in a separate data connection project, not in the agent itself. This separation means multiple agents can share the same tools without duplication.

A tool provider is a class that returns `IReadOnlyList<AITool>` using `AIFunctionFactory.Create`. Each method uses `[Description]` attributes so the LLM understands what the tool does and what each parameter means.

```
public sealed class MarketDataToolProvider { private readonly IHttpClientFactory
_httpClientFactory; private readonly ILogger<MarketDataToolProvider> _logger; public
MarketDataToolProvider(IHttpClientFactory httpClientFactory, ILogger<MarketDataToolProvider>
logger) { _httpClientFactory = httpClientFactory; _logger = logger; } public
IReadOnlyList<AITool> GetTools() { return [ AIFunctionFactory.Create(EdgarCompanyFactsAsync,
"edgar_company_facts", "Fetch SEC EDGAR financial data for a company by ticker symbol"),
AIFunctionFactory.Create(StockQuoteAsync, "stock_quote", "Get current stock price and daily
change from Yahoo Finance"), AIFunctionFactory.Create(StockHistoryAsync, "stock_history", "Get
historical price data from Yahoo Finance"), ]; } [Description("Fetch SEC EDGAR financial data
for a company by ticker symbol")] private async Task<string> EdgarCompanyFactsAsync(
```

```
[Description("Stock ticker symbol (e.g. AAPL, MSFT)")] string ticker, CancellationToken ct) {
// Implementation: call SEC EDGAR API, parse response, return JSON summary } [Description("Get
current stock price and daily change from Yahoo Finance")] private async Task<string>
StockQuoteAsync( [Description("Stock ticker symbol")] string ticker, CancellationToken ct) {
// Implementation: call Yahoo Finance API } [Description("Get historical price data from Yahoo
Finance")] private async Task<string> StockHistoryAsync( [Description("Stock ticker symbol")]
string ticker, [Description("Time range: 5d, 1mo, 3mo, 6mo, 1y, 5y")] string range,
CancellationToken ct) { // Implementation: call Yahoo Finance historical data API } }
```

The tool provider is registered in DI as `IReadOnlyList<AITool>`:

```
services.AddSingleton<MarketDataToolProvider>();
services.AddSingleton<IReadOnlyList<AITool>>(sp =>
sp.GetRequiredService<MarketDataToolProvider>().GetTools());
```

3. An Agent Tools Seed

The agent tools seed declares which tools the agent depends on by name. This creates an explicit binding between the agent and its tools that the platform validates at startup. If a declared tool is not registered, the platform logs a warning.

```
services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed( agentName: "financial-analyst",
toolNames: ["stock_quote", "stock_history", "edgar_company_facts"]));
```

The interface:

```
public interface IAgentToolsSeed { string AgentName { get; } IReadOnlyList<string> ToolNames {
get; } }
```

This is a declaration, not a restriction. The platform uses it for startup validation and tool routing. The agent can still use platform-provided tools (context management, file operations, etc.) that are registered globally.

4. Context Seeds (Optional)

Beyond the persona, you can seed additional context domains. Each domain has its own seed interface, but they all follow the same pattern: implement the interface, register it as a singleton, and the platform persists it on first startup.

Guardrail seed -- policy rules the LLM must follow:

```
services.AddSingleton<IGuardrailSeed>(new GuardrailSeed( name: "investment-disclaimer",
severity: "high", enabled: true, rule: "" Never provide specific buy, sell, or hold
recommendations. Always include a disclaimer that the analysis is informational only and not
investment advice. Do not predict future stock prices or guarantee returns. ""));
```

Context seed -- background knowledge injected into the system prompt:

```
services.AddSingleton<IContextSeed>(new ContextSeed( topic: "financial-analysis-methodology",
tags: "finance,methodology", type: "fact", content: "" ## Financial Analysis Reference -
**P/E Ratio**: Price / Earnings per Share. Compare within sector. - **Revenue Growth**:
Year-over-year revenue change. - **EPS**: Earnings per Share. Primary driver of stock
valuation. ""));
```

The full set of optional seed interfaces:

Interface	Purpose
`IGuardrailSeed`	Behavioral constraints with severity levels
`IContextSeed`	Typed knowledge (fact, reference, decision, feedback)
`IWorkflowSeed`	Multi-step workflow definitions
`IEntitySeed`	Domain entity schemas
`ISkillSeed`	Reusable prompt templates
`IUserProfileSeed`	User role, timezone, and preferences
`IMcpConnectionSeed`	MCP servers to auto-connect at startup

You can register multiple seeds of each type. Each one becomes a markdown file in the agent's data directory.

5. DI Registration

All seeds and tool registrations go into a single extension method. This is the agent's entire public API -- one method that wires everything into the service collection.

```
using AiAgentCanvas.Abstractions; using Microsoft.Extensions.DependencyInjection; namespace
Agent.FinancialAnalyst; public static class FinancialAnalystServiceExtensions { public static
IServiceCollection AddFinancialAnalystAgent(this IServiceCollection services) { // Persona
services.AddSingleton<IPersonaSeed>(new PersonaSeed( name: "financial-analyst", description:
"A financial analyst that uses market data tools", instructions: "...")); // Context,
guardrails, workflows, entities, skills, profile... services.AddSingleton<IContextSeed>(new
ContextSeed(...)); services.AddSingleton<IGuardrailSeed>(new GuardrailSeed(...));
services.AddSingleton<IWorkflowSeed>(new WorkflowSeed(...));
services.AddSingleton<IEntitySeed>(new EntitySeed(...)); services.AddSingleton<ISkillSeed>(new
SkillSeed(...)); services.AddSingleton<IUserProfileSeed>(new UserProfileSeed(...)); // Tool
binding services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed( agentName:
"financial-analyst", toolNames: ["stock_quote", "stock_history", "edgar_company_facts"]));
return services; } }
```

The agent project only needs two dependencies: `AiAgentCanvas.Abstractions` (for the seed interfaces) and `Microsoft.Extensions.DependencyInjection.Abstractions` (for `IServiceCollection`). No AI framework references needed -- the agent does not own tools or the LLM client.

6. Composition Root

One line in `Program.cs` activates the agent:

```
builder.Services.AddFinancialAnalystAgent();
```

That's it. The platform resolves all `IPersonaSeed`, `IContextSeed`, and other seed services at startup, persists them to disk, and makes them available to the runtime agent.

What the Platform Provides

Everything outside those six pieces is handled by the platform:

- **HarnessAgent runtime** -- the MAF SDK's `AsHarnessAgent` creates the agent with system prompt assembly, tool routing, and conversation management
- **AG-UI streaming** -- real-time streaming of agent responses to the frontend via the Agent-User Interaction protocol
- **Context injection** -- `AIContextProvider` implementations inject personas, guardrails, entities, profiles, and context into the system prompt before every LLM call
- **Tool deduplication** -- multiple registrations of the same tool are deduplicated automatically
- **Governance wrapping** -- `GovernedAIFunction` and `GovernedMcpGateway` evaluate every tool call against active guardrails and policies
- **File workspace** -- `FileSystemAgentFileStore` gives the agent sandboxed file read/write access
- **Background agents** -- long-running tasks execute in background agent threads
- **Tool approval** -- `ToolApprovalAgentOptions` with configurable auto-approval rules
- **Telemetry** -- `OpenTelemetry` tracing and structured logging across the full request pipeline
- **Health checks** -- readiness and liveness probes for orchestrator health
- **A2A protocol** -- Agent-to-Agent communication for multi-agent scenarios
- **Sessions and history** -- conversation persistence and session management

6. Context Domains

Context domains are the agent's knowledge system. Each domain stores a specific type of information as markdown files with YAML frontmatter. The platform injects this data into the system prompt before every LLM request, so the agent always has access to its configuration, policies, and accumulated knowledge. The agent can also manage domain data at runtime through built-in tools.

The Three-Class Pattern

Every domain follows the same structure:

- **Store** -- reads and writes markdown files from layered directories. Handles CRUD operations and file parsing via the `MarkdownFile` utility.
- **Tool provider** -- exposes the store's operations as `AITool` instances so the LLM can create, read, update, and delete domain data through conversation.
- **Context provider** -- an `AIContextProvider` that reads domain data from the store and injects it into the system prompt before each LLM call.

This pattern means data flows in both directions: the platform injects domain data *into* the LLM's context, and the LLM can modify domain data *through* tool calls. A user can say "add a guardrail that prevents sharing internal pricing" and the agent will create a guardrail file that takes effect on the next request.

The Six Domains

Domain	What It Stores	System Prompt Injection
Personas	Agent identities with name, description, and behavioral instructions	Active persona's instructions appended directly to the system prompt
Context	Typed knowledge entries: fact, reference, decision, or feedback	All entries grouped by type under a <code>### Persistent Context`</code> section
Entities	Structured records describing people, companies, projects, or other domain objects	Entity index injected under <code>### Known Entities`</code>
Guardrails	Policy rules with severity levels (critical, warning, info) and an enabled flag	Active rules injected under <code>### Guardrails & Policies`</code>
Profiles	User-specific settings: role, timezone, and freeform preferences	Active profile injected under <code>### Active User Profile`</code>

Domain	What It Stores	System Prompt Injection
Workflows	Multi-step process definitions with descriptions and tags	Not injected into the system prompt; executed on demand when the user or agent triggers them

Each domain stores its data as markdown files with YAML frontmatter. For example, a guardrail file looks like:

```
--- name: investment-disclaimer severity: high enabled: true --- Never provide specific buy, sell, or hold recommendations. Always include a disclaimer that the analysis is informational only and not investment advice.
```

Data Storage Structure

Agent data lives under a common agent-data/ root. Each agent gets its own subdirectory, with a shared/ directory for cross-agent data. Within each subdirectory, agent/ holds system-written files (seeds and tool-created content) and user/ holds hand-written files that the system reads but never overwrites.

```
agent-data/ ■■■ orchestrator/ ■ ■■■ agent/ # System writes here (seeds + tool-created content) ■ ■ ■■■ personas/ ■ ■ ■■■ context/ ■ ■ ■■■ workflows/ ■ ■ ■■■ entities/ ■ ■ ■■■ profiles/ ■ ■ ■■■ guardrails/ ■ ■■■ user/ # Hand-written files (read-only to system) ■ ■■■ personas/ ■ ■■■ context/ ■ ■■■ workflows/ ■ ■■■ entities/ ■ ■■■ profiles/ ■ ■■■ guardrails/ ■■■ shared/ # Accessible to all agents ■■■ agent/ ■ ■■■ personas/ ■ ■■■ context/ ■ ■■■ ... ■■■ user/ ■■■ personas/ ■■■ context/ ■■■ ...
```

The agent/ directory is where seed data lands on first startup and where tool-created content is saved. The user/ directory is a no-code alternative to implementing seed interfaces: drop a properly formatted markdown file into the right subdirectory and the platform picks it up automatically.

Layered Reading

Each store reads from multiple directories and merges the results. When the platform calls ListAll() on a domain store, it collects files from:

- The agent's own agent/ directory (system-written)
- The agent's own user/ directory (hand-written)
- The shared/agent/ directory (shared system-written)
- The shared/user/ directory (shared hand-written)

All files are merged into a single list. If two files have the same name, the agent-level file takes precedence over the shared-level file. This layering lets you define shared defaults (company-wide guardrails, common entity schemas) that individual agents can override with their own versions.

The context provider then reads this merged list and formats it into the appropriate system prompt section. The LLM sees user-provided, system-created, and shared data together, without needing to know where each piece came from.

How Domains Interact

Domains are independent but complementary. A typical agent configuration might include:

- A **persona** that defines the agent's role and references specific tools
- **Guardrails** that constrain the persona's behavior (e.g., "never share internal pricing")
- **Context** entries that provide background knowledge the persona needs
- **Entities** that define the domain objects the agent works with
- A **profile** that captures the user's preferences and role
- **Workflows** that define reusable multi-step processes the agent can execute

All of these are injected into the system prompt together, giving the LLM a complete picture of who it is, what it knows, what rules it must follow, and what structured processes it can run.

7. Expanding with MCP and RAG

Agents start with the tools and knowledge you seed at build time. MCP and RAG extend the agent at runtime -- MCP connects external tools and services without writing code, and RAG enriches every response with relevant documents from a vector store.

Model Context Protocol (MCP)

What MCP Does

MCP is an open standard for connecting AI agents to external tools and data sources at runtime. Instead of writing a tool provider class and redeploying, you point the agent at an MCP server endpoint and it discovers the available tools automatically. The tools appear alongside built-in tools and the LLM can call them like any other function.

Connecting via Agent Tools

The agent exposes three MCP management tools through the `McpConnectionManager`:

```
connect_mcp_server(name, endpoint, transport) -- connect and discover tools
disconnect_mcp_server(name) -- disconnect and remove tools
list_mcp_connections() -- list active connections
```

A user can connect to an MCP server through conversation:

"Connect to the GitHub MCP server at <https://mcp.github.com/sse>"

The agent calls `connect_mcp_server` with the name, endpoint, and transport type. From that point on, the GitHub server's tools are available in the conversation.

How Connection Works

The `McpConnectionManager` handles the full connection lifecycle:

- The LLM calls `connect_mcp_server` with a name, endpoint URL, and transport type
- The manager creates a transport client based on the specified type (stdio, SSE, or streamable HTTP)
- It connects via `McpClient.CreateAsync()` and calls `ListToolsAsync()` to discover the server's tools
- Discovered tools are registered into the `DynamicToolRegistry` under the key `mcp:{name}`
- The `DynamicToolContextProvider` picks up the new tools on the next LLM invocation

```
private async Task<string> ConnectMcpServer(string name, string endpoint, string transport,
Cancellation token ct) { IClientTransport clientTransport = transport.ToLowerInvariant() switch
{ "http" or "sse" => new HttpClientTransport( new HttpClientTransportOptions { Endpoint = new
Uri(endpoint) } ), _ => throw new ArgumentException($"Unsupported transport: {transport}"), };
var client = await McpClient.CreateAsync(clientTransport, cancellation token: ct); var mcpTools
= await client.ListToolsAsync(cancellation token: ct); // Register discovered tools into the
dynamic tool registry _dynamicToolRegistry.Register($"mcp:{name}", mcpTools); return
$"Connected to '{name}' -- {mcpTools.Count} tools discovered."; }
```

Once connected, MCP tools appear alongside built-in tools with no distinction from the LLM's perspective. The agent can call them, and they show up in tool listings.

Governance

Every MCP tool call passes through the `GovernedMcpGateway` before execution. The gateway evaluates the call against the agent's active guardrails and policies:

```
public McpGatewayDecision Evaluate(string agentId, string toolName, string? payload = null) {
    var request = new McpGatewayRequest { AgentId = agentId, ToolName = toolName, Payload =
    payload ?? "", }; var decision = _gateway.ProcessRequest(request); if (!decision.Allowed) {
    _logger.LogWarning( "[GOVERNANCE:MCP] Blocked tool={Tool} agent={Agent} status={Status}",
    toolName, agentId, decision.Status); } return decision; }
```

This means an MCP tool that violates a guardrail is blocked the same way a built-in tool would be. The governance layer does not distinguish between tool sources.

Building an MCP Connection Seed

To auto-connect to an MCP server at startup (instead of requiring the user to connect manually), implement `IMcpConnectionSeed`:

```
services.AddSingleton<IMcpConnectionSeed>(new McpConnectionSeed( name: "company-data",
    endpoint: "https://mcp.internal.example.com/sse", transport: "sse"));
```

The interface:

```
public interface IMcpConnectionSeed { string Name { get; } string Endpoint { get; } string
    Transport { get; } // "sse", "http", or "stdio" }
```

When the platform starts, it resolves all `IMcpConnectionSeed` services and connects to each one automatically. The agent is ready to use those tools from the first request, with no user intervention.

Retrieval-Augmented Generation (RAG)

What RAG Does

RAG retrieves relevant document chunks at query time and injects them into the agent's context. Instead of fine-tuning a model on your documents or cramming everything into the system prompt, RAG searches a vector store for the chunks most relevant to the user's current question and includes only those. The agent gets grounded, up-to-date information with source citations.

The Pipeline

Documents flow through an ingestion pipeline that prepares them for search, and queries flow through a retrieval pipeline that finds and ranks the best matches:

```
Ingestion Retrieval ===== Raw Document User Message | | v v DocumentChunker
Embed Query (recursive paragraph/sentence split) (IEmbeddingGenerator) | | v v Embed Each
Chunk Hybrid Search (IEmbeddingGenerator) (vector cosine + FTS5 keyword) | | v v Store in
SQLite LLM Reranking (vector BLOB + FTS5 index) (top-10 candidates -> top-3) | v Citation
Formatting ([1] source: X, score: 0.82) | v Inject into Agent Context
```

Chunking

The DocumentChunker recursively splits documents into chunks suitable for embedding. It follows a semantic hierarchy rather than using fixed-size character windows:

- **Split by paragraphs** (double newlines)
- If a paragraph exceeds ChunkSize, **split by sentences** (period, exclamation, or question mark followed by space or newline)
- Apply **overlap** between consecutive chunks for context continuity
- Discard chunks under 20 characters

```
public sealed class DocumentChunker { public int ChunkSize { get; init; } = 512; public int ChunkOverlap { get; init; } = 64; public List<DocumentChunk> Chunk(string text, string? source = null) { // 1. Split by paragraphs // 2. If paragraph > ChunkSize, split by sentences // 3. Apply overlap between consecutive chunks // 4. Filter out chunks < 20 chars } }
```

Property	Default	Description
`ChunkSize`	512	Maximum characters per chunk
`ChunkOverlap`	64	Characters of overlap between consecutive chunks

Embedding

Each chunk is embedded into a vector using Azure AI Foundry's embedding model. The platform uses IEmbeddingGenerator from Microsoft.Extensions.AI, so the embedding provider is swappable via DI. The same embedding model is used for both document ingestion and query-time embedding to ensure consistent vector space alignment.

Storage

Chunks are stored as DocumentRecord entries in a SQLite database with two indexing strategies:

- **Vector BLOB** -- the embedding vector stored as a binary blob for cosine similarity search
- **FTS5 full-text index** -- the chunk text indexed with SQLite's FTS5 extension for keyword/BM25 search

This dual storage enables hybrid search without requiring a separate search engine.

Hybrid Search

The retrieval pipeline combines two search signals:

- **Vector search** (cosine similarity) -- captures semantic meaning. A query about "company revenue" matches chunks discussing "annual income" or "top-line growth" even without exact keyword overlap.
- **Keyword search** (SQLite FTS5 / BM25) -- captures exact term matches. Important for proper nouns, ticker symbols, product names, and technical terms that semantic search alone might miss.

The scores from both signals are combined to produce a unified ranking. This hybrid approach consistently outperforms either signal alone.

LLM Reranking

After hybrid search returns the top candidates (typically 10), the platform sends them to the LLM for relevance reranking. The LLM evaluates each candidate against the original query and assigns a relevance score. Only the top-K survivors (typically 3) are kept for injection.

This step is expensive (it costs an additional LLM call) but significantly improves precision. The initial retrieval stage optimizes for recall -- casting a wide net -- while reranking optimizes for precision -- keeping only the truly relevant chunks.

Context Injection

Surviving chunks are formatted with citation numbers and injected into the agent's context:

```
[1] source: quarterly-report-q3.pdf, score: 0.92 Revenue grew 15% year-over-year to $2.1B, driven primarily by cloud services... [2] source: earnings-call-transcript.pdf, score: 0.87 The CEO noted that enterprise customer count increased by 23% in Q3...
```

The agent can reference these citations in its response, giving users a clear trail back to the source documents.

Enabling RAG

RAG is opt-in and requires an embedding model deployment. Set the deployment name in configuration:

```
{ "AIFoundry": { "EmbeddingDeploymentName": "text-embedding-ada-002" } }
```

When this value is set, the platform auto-registers all RAG components at startup:

- DocumentChunker for splitting documents
- IEmbeddingGenerator configured against the Azure AI Foundry embedding deployment
- SQLite vector store with FTS5 indexing
- Hybrid search and reranking pipeline
- RAG context provider that injects retrieved chunks before each LLM call

When the value is not set, RAG components are not registered and the agent operates without document retrieval. No code changes needed either way -- the presence of the configuration key is the only switch.

8. Multi-Agent Setup

AI Agent Canvas supports three topologies for running multiple agents. Each offers different trade-offs between simplicity, isolation, and scalability. You can combine them -- use in-process handoff and background delegation within a single host, then connect hosts with A2A when you need independent deployment.

Option 1: In-Process Handoff

All agents run in the same .NET process. The main agent delegates to a specialist by calling the `handoff_to_agent` tool. The specialist runs to completion and returns its response directly.

Under the hood, the flow is:

- `HandoffToolProvider` exposes the `handoff_to_agent` tool to the LLM
- `InProcessAgentHandoff` receives the call with the target agent name and context
- `AgentRegistry.Resolve` finds or builds the target agent (loading its persona and scoped tools)
- The target agent runs as a `ChatClientAgent` with its own persona instructions
- The response flows back to the calling agent as a tool result

When to use: Multiple specialists without deployment complexity. Shared LLM endpoint. Low latency between agents. All agents owned by the same team.

Setup

Register a persona seed and tool seed for each specialist agent. The `AgentRegistry` builds agents on demand from these seeds.

```
public static IServiceCollection AddFinancialAnalyst(this IServiceCollection services) { //
Persona seed defines the agent's identity and instructions
services.AddSingleton<IPersonaSeed>(new PersonaSeed( name: "financial-analyst", description:
"A financial analyst that uses market data tools to research stocks and generate reports.",
instructions: "You are a financial analyst assistant. Use the available market data tools to
research stocks, analyze trends, and generate investment reports.")); // Tool seed restricts
which tools this agent can see
services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed(
agentName: "financial-analyst", toolNames: ["stock_quote", "stock_history",
"edgar_company_facts"])); return services; }
```

The handoff tool is registered automatically by `HandoffToolProvider`. Any persona in the registry is available as a handoff target:

```
// HandoffToolProvider exposes this to the LLM [Description("Delegate a task to another agent
and get the result back.")] private async Task<string> HandoffToAgent(string targetAgent,
string context) { var result = await _handoff.HandoffAsync(targetAgent, context); return
JsonSerializer.Serialize(result); }
```

When the LLM decides to delegate, it calls `handoff_to_agent` with the target name and a context message. The specialist runs in the same process, and the result comes back as a tool response.

Option 2: Background Delegation

The main agent spawns parallel tasks on background agents. This is asynchronous -- the agent can start multiple tasks, continue working on other things, and collect results when they finish.

Background delegation exposes five tools to the LLM:

Tool	Purpose
`background_agents_start_task`	Start a task on a named background agent
`background_agents_wait_for_first_completion`	Block until the first background task finishes
`background_agents_get_task_results`	Get the result of a specific completed task
`background_agents_get_all_tasks`	List all background tasks and their status
`background_agents_continue_task`	Send a follow-up message to a running task

When to use: Independent parallel tasks (research multiple topics simultaneously). Synthesizing results from multiple specialists. Tasks where the LLM decides at runtime which agents to involve.

Setup

Background delegation is automatic. Any persona registered in AgentRegistry is available as a background agent. The platform resolves all non-default agents and passes them to the HarnessAgent:

```
var backgroundAgents = registry.ListAvailableAgents().Where(n => !n.Equals("default",
StringComparison.OrdinalIgnoreCase)).Select(n => registry.Resolve(n)).Where(a => a is not
null).ToList(); AIAgent agent = chatClient.AsHarnessAgent(new HarnessAgentOptions { Name =
options.AgentName, BackgroundAgents = backgroundAgents is { Count: > 0 } ? backgroundAgents :
null, // ... });
```

No additional configuration needed. Register persona and tool seeds, and the agent becomes available for both handoff and background delegation.

Example: Parallel Research

The LLM might use background delegation like this during a conversation:

```
User: Compare the financial health of Apple, Microsoft, and Google. Agent (internal
reasoning): -> background_agents_start_task("financial-analyst", "Analyze Apple's financial
health using AAPL data") -> background_agents_start_task("financial-analyst", "Analyze
Microsoft's financial health using MSFT data") ->
background_agents_start_task("financial-analyst", "Analyze Google's financial health using
GOOGL data") -> background_agents_wait_for_first_completion() -> (collects all three results)
-> Synthesizes a comparative report
```

Option 3: A2A Protocol

Separate hosts communicating over HTTP. Each host exposes an /a2a endpoint and publishes AgentCard metadata describing its capabilities. Remote agents behave like local ones -- handoff and background tools work transparently.

When to use: Independent deployment and scaling per agent. Different security boundaries. Different teams owning different agents. Third-party agents from external organizations.

Setup

Each host runs its own instance of AI Agent Canvas with its own agents. The host exposes the A2A endpoint:

```
// In Program.cs of the remote agent host var app = builder.Build(); app.UseAiAgentCanvas();
app.MapA2AEndpoints(agentName: "ComplianceAgent", path: "/a2a"); app.Run();
```

The A2A server is registered in DI:

```
services.AddA2AServer(agentName);
```

The orchestrator registers remote agents by URL. They join the registry and become available through the same handoff and background tools:

```
// In the orchestrator's startup var registry =
app.Services.GetRequiredService<AgentRegistry>(); registry.RegisterRemote( name:
"compliance-checker", a2aUrl: "https://compliance-service.internal/a2a", description: "Reviews
documents for regulatory compliance");
```

The RegisterRemote method creates an AgentCard, configures an HttpClient with the remote URL, and builds an agent proxy:

```
public void RegisterRemote(string name, string a2aUrl, string? description = null) { var card
= new AgentCard { Name = name, Description = description ?? $"Remote A2A agent at {a2aUrl}",
Version = "1.0" }; var httpClient = _httpClientFactory?.CreateClient(name) ?? new
HttpClient(); httpClient.BaseAddress = new Uri(a2aUrl); var agent = card.AsIAgent(httpClient,
loggerFactory: _loggerFactory); _agents[name] = agent; }
```

After registration, handoff_to_agent("compliance-checker", ...) sends the request over HTTP to the remote host. The calling agent does not need to know whether the target is local or remote.

Comparison

	In-Process Handoff	Background Delegation	A2A Protocol
Latency	Lowest (in-memory)	Low (in-memory, async overhead)	Higher (HTTP round-trip)
Isolation	None (shared process)	None (shared process)	Full (separate hosts)
Parallelism	Sequential only	Parallel tasks	Parallel via separate hosts
Scaling	Single process	Single process	Independent per host
Complexity	Low	Low	Moderate (deployment, networking)
Security boundary	Shared	Shared	Independent (separate auth, policies)

Composing Topologies

These options are not mutually exclusive. A practical setup might look like:

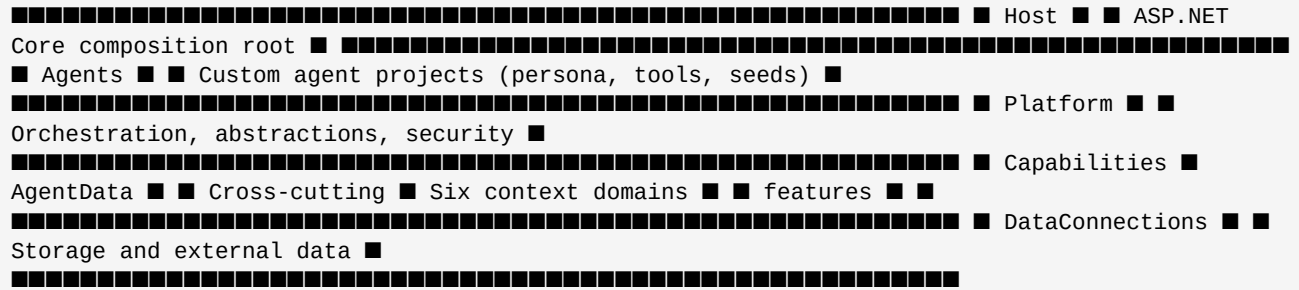
- **In-process + background** for your team's agents: a financial analyst, a report generator, and a data researcher all running in one host, using handoff for sequential work and background delegation for parallel tasks.
- **A2A** for other teams' agents: the compliance team runs their own host with their own deployment schedule and security policies. Your orchestrator connects to it via `RegisterRemote`.

The agent code does not change between topologies. A persona that works as an in-process handoff target works identically as a remote A2A agent. The difference is in how you register it.

9. Architecture

The platform is organized into five layers. Each layer depends only on the layers below it. This constraint keeps the dependency graph clean -- you can change a capability without touching the platform, or swap a data connection without affecting agent logic.

Layer Overview



Layer	What It Does
Host	ASP.NET Core composition root. Wires all layers together, runs the web server, maps HTTP endpoints. The only project that references everything else.
Agents	Custom agent projects. Each registers its persona, tools, and seeds via an extension method. Agent projects reference only Platform.Abstractions.
Platform	Orchestration engine, shared abstractions, and security. Contains the agent runtime, registry, handoff, AG-UI server, A2A server, and governance.
Capabilities	Cross-cutting features that any agent can use. Skills, scheduling, notifications, system tools, and RAG.
AgentData	Six context domains that store and provide agent knowledge: personas, context, entities, guardrails, profiles, and workflows.
DataConnections	Storage backends and external data sources. SQLite for persistence, vector store for RAG, market data for domain-specific tools.

Project Reference Map

```

Host ■■■ Platform.Orchestration ■ ■■■ Platform.Abstractions ■■■ Platform.Security ■ ■■■
Platform.Abstractions ■■■ Agent.FinancialAnalyst ■ ■■■ Platform.Abstractions ■■■
Capabilities.Skills ■ ■■■ Platform.Abstractions ■ ■■■ Platform.Orchestration ■■■
Capabilities.Scheduling ■ ■■■ Platform.Abstractions ■ ■■■ AgentData.Personas ■■■
Capabilities.Notifications ■ ■■■ Platform.Abstractions ■■■ Capabilities.SystemTools ■ ■■■
Platform.Abstractions ■■■ Capabilities.Rag ■ ■■■ Platform.Abstractions ■■■
AgentData.Personas ■ ■■■ Platform.Abstractions ■■■ AgentData.Context ■ ■■■
Platform.Abstractions ■■■ AgentData.Entities ■ ■■■ Platform.Abstractions ■■■
AgentData.Guardrails ■ ■■■ Platform.Abstractions ■■■ AgentData.Profiles ■ ■■■
Platform.Abstractions ■■■ AgentData.Workflows ■ ■■■ Platform.Abstractions ■ ■■■
Platform.Orchestration ■■■ DataConnections.Storage.Sqlite ■ ■■■ Platform.Abstractions ■■■
DataConnections.VectorStore.Sqlite ■ ■■■ Platform.Abstractions ■■■
DataConnections.MarketData ■ ■■■ Platform.Abstractions ■■■ Providers.AzureAIFoundry

```

Projects in Detail

Platform Layer

The platform provides the runtime that powers every agent. It is split into three projects to separate concerns.

Project	Purpose	Key Types
Platform.Abstractions	Interfaces and contracts. Every other project references this. No implementations.	<code>`IAgentRegistry`</code> , <code>`IAgentHandoff`</code> , <code>`IAgentMessaging`</code> , <code>`IPersonaSeed`</code> , <code>`IAgentToolsSeed`</code> , <code>`IContextSeed`</code> , <code>`IWorkflowSeed`</code> , <code>`IEntitySeed`</code> , <code>`IGuardrailSeed`</code> , <code>`ISkillSeed`</code> , <code>`IUserProfileSeed`</code> , <code>`IGoalSeed`</code> , <code>`IMcpConnectionSeed`</code> , <code>`ToolStateMapping`</code>
Platform.Orchestration	Agent runtime and coordination. Builds agents from seeds, manages handoff, runs the AG-UI and A2A servers.	<code>`AgentRegistry`</code> , <code>`InProcessAgentHandoff`</code> , <code>`InProcessAgentMessaging`</code> , <code>`HandoffToolProvider`</code> , <code>`ToolDeduplicatingChatClient`</code> , <code>`CoreServiceExtensions`</code>
Platform.Security	Governance and policy enforcement. Wraps tool calls with policy checks, injects security context.	<code>`GovernedAIFunction`</code> , <code>`GovernanceToolWrapper`</code> , <code>`GovernedMcpGateway`</code> , <code>`GovernanceContextProvider`</code>

AgentData Layer

Six domain-specific projects that store and provide agent knowledge. Each follows the same pattern: a store for persistence, tools for LLM access, and a context provider for automatic injection.

Domain	Store	Tools	Context Provider
Personas	`PersonaStore`	`PersonaToolProvider` (list, read, set active)	`PersonaContextProvider` -- injects active persona instructions
Context	`PersistentContextStore`	`ContextToolProvider` (add, list, remove)	`PersistentContextProvider` -- injects all stored context entries
Entities	`EntityStore`	`EntityToolProvider` (create, read, update, list)	`EntityContextProvider` -- injects entity index
Guardrails	`GuardrailStore`	`GuardrailToolProvider` (list, read, set active)	`GuardrailContextProvider` -- injects active guardrail rules
Profiles	`UserProfileStore`	`UserProfileToolProvider` (read, update)	`UserProfileContextProvider` -- injects user profile context
Workflows	`WorkflowStore`	`WorkflowToolProvider` (create, list, run, run_sequential, run_concurrent)	None (workflows are invoked explicitly, not injected)

Capabilities Layer

Cross-cutting features that agents use but that are not specific to any single domain.

Capability	What It Provides	Key Types
Skills	Reusable, parameterized procedures agents can invoke by name	`SkillStore`, `SkillToolProvider`, `SkillExecutor`
Scheduling	Cron-based and one-time scheduled agent tasks	`ScheduleStore`, `ScheduleToolProvider`, `ScheduleExecutor`
Notifications	Agent-to-user notification delivery	`NotificationStore`, `NotificationToolProvider`
SystemTools	General-purpose tools: date/time, math, web search, file operations	`SystemToolProvider`
RAG	Vector-based retrieval-augmented generation with citation tracking	`RagToolProvider`, `RagContextProvider`, `VectorSearchService`

DataConnections and Providers

Storage backends and external data sources. These projects implement the persistence and data-access interfaces defined in Abstractions.

DataConnections:

Project	Purpose	Key Types
Storage.Sqlite	SQLite-backed persistence for all stores (personas, entities, context, schedules, etc.)	`SqliteStorageProvider`, migration scripts
VectorStore.Sqlite	SQLite-backed vector store for RAG embeddings	`SqliteVectorStore`
MarketData	Stock quotes, price history, and SEC EDGAR company filings	`StockQuoteTool`, `StockHistoryTool`, `EdgarCompanyFactsTool`

Providers:

Project	Purpose
Providers.AzureAIFoundry	LLM client configuration for Azure AI Foundry (Azure OpenAI) endpoints

Frontend

The frontend is a Next.js 15 application using React 19. It connects to the backend via the AG-UI protocol -- a raw SSE (Server-Sent Events) client that consumes the event stream directly.

There is no CopilotKit SDK dependency. The frontend implements the AG-UI client from scratch, handling event types like `TEXT_MESSAGE_CONTENT`, `TOOL_CALL_START`, `TOOL_CALL_END`, `STATE_SNAPSHOT`, and `STATE_DELTA` to render the chat interface, tool activity, and live state updates.

Request Flow

A user message travels through the system in this sequence:

- **Frontend POSTs to** `/api/copilotkit` -- the AG-UI endpoint. The request contains the user message, conversation history, and frontend state.
- **AG-UI server resolves the session** -- identifies the user, loads conversation history, and prepares the agent context.
- **Context providers inject data** -- each registered `AIContextProvider` appends its domain knowledge to the system prompt: persona instructions, guardrail rules, entity index, user profile, RAG results, and governance context.
- **HarnessAgent runs the agent loop** -- the agent built via `AsHarnessAgent` executes the reason-act-observe cycle. The LLM receives the assembled prompt with all context and tool definitions.

10. Agent Behavior Patterns

Agent behavior patterns describe how an agent reasons, decides, and interacts. The platform does not enforce a pattern -- the pattern emerges from how you write the persona instructions, which tools you expose, and how you wire up inter-agent communication. The same platform primitives support all seven patterns.

Overview

Pattern	Key Mechanism	When to Use
Sequential	Persona instructions with ordered steps	Linear analysis pipelines, ETL, report generation
Reactive (ReAct)	Tool loop (built-in)	Open-ended tasks where the agent decides what to do next
Parallel (Fan-out/Fan-in)	Background agents	Independent sub-tasks that can run concurrently
Deliberate (Planning)	Workflow definitions	Complex multi-step tasks with predefined procedures
Reflective (Self-Critique)	Critic agent or self-review step	High-stakes outputs needing quality verification
Hierarchical (Delegation)	Handoff to worker agents	Complex problems with clear domain boundaries
Collaborative (Multi-Agent Debate)	Agent messaging	Decisions benefiting from diverse viewpoints

1. Sequential

The agent follows a fixed sequence of steps defined in its persona instructions. Each step runs to completion before the next begins. The LLM does not decide the order -- the instructions dictate it.

When to use: Predictable multi-step workflows where the order is known upfront. Report generation, data extraction pipelines, onboarding checklists.

How to implement: Define the steps in the persona seed's instructions. The agent's tool loop follows them in order.

```
services.AddSingleton<IPersonaSeed>(new PersonaSeed( name: "daily-report-generator",
description: "Generates daily market reports following a fixed procedure.", instructions: ""
You generate a daily market report. Follow these steps in order: 1. Use `stock_quote` to get
the current price for AAPL, MSFT, and GOOGL 2. Use `stock_history` to get the 5-day price
history for each 3. Use `edgar_company_facts` to get the latest quarterly revenue for each 4.
Write a summary report with price trends and revenue comparison 5. Save the report using
`add_context` with key "daily-report-{date}" Do not skip steps. Do not reorder steps. ""));
```

The LLM sees these instructions in its system prompt and executes the tool calls in the specified order.

2. Reactive (ReAct)

The default behavior pattern. The LLM reasons about the current situation, decides which tool to call, observes the result, and repeats until the goal is met. This is the standard reason-act-observe loop that every agent runs by default.

When to use: Most tasks. Any situation where the agent needs to decide at runtime what to do next based on the current state.

How to implement: No special configuration. Every agent uses this pattern by default. The `FunctionInvokingChatClient` handles the loop automatically:

```
// The reactive loop is built into the agent runtime
AIAgent agent =
chatClient.AsHarnessAgent(new HarnessAgentOptions { Name = "assistant", ChatOptions = new
ChatOptions { Instructions = "You are a helpful assistant.", Tools = tools, }, });
```

The LLM receives all available tools and decides which to call based on the user's request. If the first tool call does not produce a satisfactory result, the LLM reasons about what to try next.

3. Parallel (Fan-out/Fan-in)

The main agent spawns multiple background tasks on specialist agents, waits for them to complete, and synthesizes the results. Tasks run concurrently.

When to use: Independent sub-tasks that do not depend on each other. Researching multiple topics simultaneously. Gathering data from multiple sources in parallel.

How to implement: Use background agent tools. Any persona in the `AgentRegistry` is automatically available as a background agent.

```
LLM decides to fan out: -> background_agents_start_task("financial-analyst", "Analyze AAPL
financial health") -> background_agents_start_task("financial-analyst", "Analyze MSFT
financial health") -> background_agents_start_task("financial-analyst", "Analyze GOOGL
financial health") -> background_agents_wait_for_first_completion() ->
background_agents_get_all_tasks() // check which are done ->
background_agents_get_task_results(task_id) // collect each result -> Synthesize comparative
report from all three results
```

You can also use the workflow executor for structured parallel execution:

```
// Programmatic concurrent execution via MAF workflow builder
var agents = agentNames
.Select(name => registry.Resolve(name))
.Where(a => a is not null)
.ToList();
var workflow =
AgentWorkflowBuilder.BuildConcurrent(agents);
var input = new ChatMessage(ChatRole.User,
userInput);
await using var run = await InProcessExecution.RunAsync(workflow, input,
cancellationToken: ct);
```

4. Deliberate (Planning)

Workflow definitions decompose goals into named steps. Instead of relying on the LLM to figure out the steps at runtime, you define the procedure upfront. The agent follows the workflow, using its tools to execute each step.

When to use: Complex multi-step tasks with well-known procedures. Compliance reviews, deployment checklists, data migration pipelines.

How to implement: Create workflows using the workflow tools or seed them at startup. Three execution modes are available:

Simple workflow -- a markdown-defined procedure the agent follows:

```
services.AddSingleton<IWorkflowSeed>(new WorkflowSeed( agentName: "compliance-reviewer", name:
"quarterly-review", content: "" # Quarterly Compliance Review ## Steps 1. Pull all
transactions from the last quarter 2. Flag any transaction over $10,000 without documentation
3. Check each flagged transaction against the exceptions list 4. Generate a findings report
with severity ratings 5. Create follow-up tasks for any critical findings ""));
```

The agent calls `run_workflow("quarterly-review", "Run the Q3 2025 review")` and the executor feeds the workflow steps into the LLM as structured instructions.

Sequential workflow -- chains multiple agents in order, each receiving the previous agent's output:

```
var workflow = AgentWorkflowBuilder.BuildSequential(agents); await using var run = await
InProcessExecution.RunAsync(workflow, inputMessage, cancellationTokens: ct);
```

Declarative workflow -- YAML-defined workflow graphs loaded and executed via MAF's `DeclarativeWorkflowBuilder`:

```
var workflow = DeclarativeWorkflowBuilder.Build<ChatMessage>(filePath, options); await using
var run = await InProcessExecution.RunAsync(workflow, inputMessage, cancellationTokens: ct);
```

5. Reflective (Self-Critique)

The agent reviews its own output before delivering it, or hands off to a dedicated critic agent that evaluates quality. The review step catches errors, improves clarity, and verifies that the output meets requirements.

When to use: High-stakes outputs: legal documents, financial reports, customer-facing content. Any situation where getting it wrong is expensive.

How to implement: Two approaches.

Self-review via persona instructions:

```
services.AddSingleton<IPersonaSeed>(new PersonaSeed( name: "report-writer", description:
"Writes reports with built-in quality review.", instructions: "" After generating any report:
1. Review each claim for accuracy against the source data 2. Check that all requested sections
are present 3. Verify that numbers are consistent across sections 4. If you find errors, fix
them before delivering the final version 5. State what you checked at the end of the report
""));
```

Critic agent via handoff:

```
// Register a critic persona services.AddSingleton<IPersonaSeed>(new PersonaSeed( name:
"quality-reviewer", description: "Reviews agent outputs for accuracy and completeness.",
instructions: "" You are a quality reviewer. When given a document: 1. Check all factual
claims against available data 2. Identify any logical inconsistencies 3. Flag missing
information 4. Return a verdict: PASS, REVISE (with specific feedback), or FAIL (with reasons)
"")); // The main agent's instructions tell it to use the critic // "After generating the
report, use handoff_to_agent('quality-reviewer', report) to get a quality review."
```

6. Hierarchical (Delegation)

A manager agent decomposes a complex problem into sub-tasks and delegates each to a specialist worker agent via handoff. Workers return results. The manager synthesizes the final output.

When to use: Complex problems with clear domain boundaries. Each sub-domain has its own expertise, tools, and knowledge. The manager orchestrates but does not do the domain-specific work.

How to implement: Register specialist agents with scoped tools. The manager agent's persona instructs it to delegate:

```
// Manager agent with broad tool access services.AddSingleton<IPersonaSeed>(new PersonaSeed(
name: "project-manager", description: "Orchestrates complex projects by delegating to
specialists.", instructions: "" You are a project manager. When given a complex request: 1.
Break it into sub-tasks based on domain expertise 2. Delegate each sub-task to the appropriate
specialist using handoff_to_agent 3. Review each specialist's response for quality and
completeness 4. Synthesize a final response that integrates all results Available specialists:
- financial-analyst: market data, stock analysis, financial reports - compliance-reviewer:
regulatory checks, policy validation - report-writer: document formatting, narrative writing
"")); // Each specialist has its own scoped tools services.AddSingleton<IAgentToolsSeed>(new
AgentToolsSeed( agentName: "financial-analyst", toolNames: ["stock_quote", "stock_history",
"edgar_company_facts"])); services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed(
agentName: "compliance-reviewer", toolNames: ["list_guardrails", "read_guardrail",
"read_entity"]));
```

The manager calls `handoff_to_agent` for each sub-task and receives structured results. It then combines them into a coherent final response.

7. Collaborative (Multi-Agent Debate)

Multiple agents share perspectives on the same problem via messaging. Different personas reason about the same question from different angles. A synthesizer agent collects the perspectives and produces a balanced conclusion.

When to use: Decisions that benefit from diverse viewpoints. Risk assessments, strategy decisions, ethical reviews. Cases where a single perspective might miss important considerations.

How to implement: Use `IAgentMessaging` for asynchronous inter-agent communication:

```
// Define agents with different perspectives services.AddSingleton<IPersonaSeed>(new
PersonaSeed( name: "optimist-analyst", description: "Analyzes opportunities and upside
potential.", instructions: "Analyze the given scenario focusing on opportunities, growth
potential, and best-case outcomes. Be specific about what could go right."));
services.AddSingleton<IPersonaSeed>(new PersonaSeed( name: "risk-analyst", description:
"Analyzes risks and downside potential.", instructions: "Analyze the given scenario focusing
on risks, threats, and worst-case outcomes. Be specific about what could go wrong."));
```

The orchestrator fans out the question to both analysts (via background delegation or handoff), collects their perspectives, and synthesizes a balanced view:

```
Orchestrator: -> background_agents_start_task("optimist-analyst", "Evaluate expanding into the
European market") -> background_agents_start_task("risk-analyst", "Evaluate expanding into the
European market") -> Wait for both to complete -> Synthesize: "The optimist sees X, Y, Z. The
risk analyst flags A, B, C. Balanced recommendation: ..."
```

For more structured collaboration, agents can exchange messages directly:

```
public interface IAgentMessaging { Task SendAsync(AgentMessage message, CancellationToken
cancellationTok = default); Task<AgentMessage?> ReceiveAsync(string agentName,
CancellationToken cancellationTok = default); Task<IReadOnlyList<AgentMessage>>
ReceiveAllAsync(string agentName, CancellationToken cancellationTok = default); }
```

Tool Scoping

IAgentToolsSeed restricts which tools each agent can see. When the AgentRegistry builds an agent from a persona, it filters the global tool set to include only the tools named in that agent's seed.

```
// In AgentRegistry.BuildAgentForPersona var allTools = _toolsFactory().ToList(); var tools =
_toolSeeds.TryGetValue(persona.Name, out var seed) ? allTools.Where(t =>
seed.ToolNames.Contains(t.Name)).ToList() : allTools;
```

If no tool seed is registered for an agent, it receives all tools.

Benefits of tool scoping:

- **Focus** -- fewer irrelevant tools means the LLM spends less time reasoning about tools it should not use. A financial analyst does not need to see scheduling tools.
- **Safety** -- limit access to dangerous or sensitive tools. A read-only analyst should not have write tools.
- **Cost** -- fewer tools in the prompt means fewer tokens in tool descriptions, which reduces LLM costs on every turn.

Register a tool seed alongside the persona seed:

```
services.AddSingleton<IAgentToolsSeed>(new AgentToolsSeed( agentName: "financial-analyst",
toolNames: ["stock_quote", "stock_history", "edgar_company_facts"]));
```

Choosing a Pattern

Start with Reactive (the default) and add structure only when the task demands it.

```
Is it a single-step task? ■■ Yes -> Reactive (default tool loop) ■■ No ■■ Is the sequence of
steps known upfront? ■ ■■ Yes -> Sequential (persona instructions) or Deliberate (workflow) ■
■■ No -> Reactive ■ ■■ Are there independent sub-tasks? ■ ■■ Yes -> Parallel (background
agents) ■ ■■ Does it need domain-specific specialists? ■ ■■ Yes -> Hierarchical (manager
delegates to workers) ■ ■■ Does the output need quality verification? ■ ■■ Yes -> Reflective
(self-review or critic agent) ■ ■■ Would multiple perspectives improve the decision? ■■ Yes
-> Collaborative (multi-agent debate)
```

Patterns compose. A hierarchical system can use parallel fan-out to run workers concurrently. A deliberate workflow can include a reflective step at the end. A collaborative debate can feed into a sequential report-generation pipeline. Start simple and layer patterns as complexity grows.

Appendix A: User Guide

Getting Started

Prerequisites

- .NET 9 SDK
- Node.js 20+
- Azure AI Foundry account (or any Azure OpenAI endpoint)

Setup Steps

- Clone the repository.
- Build the backend:

```
dotnet build
```

- Configure `src/Host/AiAgentCanvas.Host/appsettings.Development.json` with your Azure AI Foundry credentials:

```
{ "AIFoundry": { "Endpoint": "https://YOUR-RESOURCE.openai.azure.com", "Key": "your-api-key",  
  "DeploymentName": "gpt-4o", "UseAzureCredential": false } }
```

- Run the backend:

```
dotnet run --project src/Host/AiAgentCanvas.Host
```

- In a separate terminal, start the frontend:

```
cd frontend npm install npm run dev
```

- Open `http://localhost:3000` in your browser.

Five Things to Try First

- **Ask a question** -- type any question in the chat box and watch the streaming response.
- **Create a persona** -- say "create a persona called Research Assistant that specializes in summarizing articles."
- **Run a skill** -- say "create a skill called Summarize that takes a URL and returns a summary," then "run skill Summarize."
- **Check a stock quote** -- say "get me a stock quote for MSFT" to exercise the market data tools.
- **Create a workflow** -- say "create a workflow called Morning Briefing that checks stocks then summarizes news."

Configuration Reference

appsettings.json Structure

The base configuration file lives at `src/Host/AiAgentCanvas.Host/appsettings.json`.

Section	Key	Type	Description
`AIFoundry`	`Endpoint`	string	Azure OpenAI endpoint URL
`AIFoundry`	`Key`	string	API key (leave blank if using Azure credential)
`AIFoundry`	`DeploymentName`	string	Chat model deployment name (e.g., `gpt-4o`)
`AIFoundry`	`EmbeddingDeploymentName`	string	Embedding model deployment; enables RAG when set
`AIFoundry`	`UseAzureCredential`	bool	Use `DefaultAzureCredential` instead of API key
`Security`	`PolicyPath`	string	Path to the governance policy YAML file
`Security`	`RateLimitPerMinute`	int	Maximum tool calls per minute (default: 30)
`VectorStore`	`ConnectionString`	string	SQLite connection string for the RAG vector store
`ApplicationInsights`	`ConnectionString`	string	Azure Monitor connection string for telemetry

Environment-Specific Overrides

ASP.NET Core loads configuration in layers. Place environment-specific values in:

- `appsettings.Development.json` -- local development settings (API keys, local endpoints).
- `appsettings.Production.json` -- production settings (real keys, production endpoints).

The environment-specific file merges on top of the base `appsettings.json`.

Environment Variable Overrides

Any configuration key can be overridden via environment variables using the double-underscore separator:

```
AIFoundry__Key=your-key-here AIFoundry__Endpoint=https://prod.openai.azure.com
Security__RateLimitPerMinute=60
```

Frontend Configuration

The frontend reads its backend URL from `frontend/.env.local`:

```
NEXT_PUBLIC_BACKEND_URL=http://localhost:5000
```

Change this to point at your deployed backend in production.

Using the Chat Interface

UI Elements

- **Streaming responses** -- text appears token by token as the model generates it.
- **Tool call indicators** -- a status bar shows which tools are running and when they complete.
- **State panel** -- displays structured data returned by tools (stock quotes, task lists, entity data).
- **Reasoning blocks** -- shows chain-of-thought when the model reasons through a problem, then fades after 5 seconds.
- **Interrupt approve/deny buttons** -- when the agent requests approval for a sensitive action, buttons appear to approve or deny.
- **Health status banner** -- displays connection and service health at the top of the page.
- **Notifications** -- server-sent events (SSE) push real-time updates for scheduled tasks, workflow completions, and errors.

Troubleshooting

Problem	Likely Cause	Solution
No response after sending a message	Backend is not running or frontend cannot reach it	Verify the backend is running on port 5000 and <code>NEXT_PUBLIC_BACKEND_URL</code> in <code>.env.local</code> is correct
Response starts then times out	Model deployment is overloaded or the API key has expired	Check the backend console for HTTP 429 or 401 errors; verify <code>AIFoundry:Key</code> and <code>AIFoundry:Endpoint</code>
Tool call shows "blocked by governance policy"	Governance policy denied the tool call	Review <code>governance-policy.yaml</code> for deny rules matching the tool; adjust the policy or use a different approach
State panel stays blank	The tool does not have a <code>ToolStateMapping</code> registered	Only tools with a registered <code>ToolStateMapping</code> emit state events; check if the tool is mapped in its service extensions
Health check failed banner appears	Backend health endpoint returned an error	Check backend logs for startup failures; verify database connectivity and API key validity

Managing Agent Data

All agent data is managed through natural language commands in the chat. Each domain supports create, read, update, and delete operations.

Personas

- "Create a persona called **Technical Writer** that writes clear, concise documentation."
- "Switch to the **Technical Writer** persona."
- "List all personas."
- "Delete the **Technical Writer** persona."

Context

- "Save a fact: our fiscal year starts in July."
- "List all context entries."
- "Delete the context entry about fiscal year."

Entities

- "Save an entity for **Acme Corp** -- they are a partner company based in Seattle."
- "Search entities for **Acme**."
- "List all entities."
- "Delete the entity **Acme Corp**."

Guardrails

- "Create a guardrail called **No PII** with severity high: never include personal identifiable information in responses."
- "Toggle the **No PII** guardrail off."
- "List all guardrails."

User Profiles

- "Create a profile called **Developer** with role engineer, timezone US/Pacific."
- "Switch to the **Developer** profile."
- "List all profiles."

Workflows

- "Create a workflow called **Daily Report** that summarizes open tasks then drafts a status email."
- "Run the **Daily Report** workflow."
- "List all workflows."
- "Delete the **Daily Report** workflow."

Skills

- "Create a skill called **Translate** that translates text to a given language."
- "Run the **Translate** skill with target language French."
- "List all skills."

Scheduling

- "Schedule a task to run the **Daily Report** workflow every weekday at 9am."
- "List all scheduled tasks."
- "Cancel the **Daily Report** scheduled task."

Reference: AG-UI Protocol

The AG-UI (Agent-User Interface) protocol defines how the frontend communicates with the agent backend. AI Agent Canvas implements this protocol using server-sent events (SSE) over a single HTTP POST endpoint.

Endpoint

```
POST /api/copilotkit Content-Type: application/json
```

The endpoint is mapped in `CoreServiceExtensions.UseAiAgentCanvas()` via `MapAGUIServer()` from the `Microsoft.Agents.AI.Hosting.AGUI.AspNetCore` package.

Request Format

```
{ "threadId": "thread_abc123", "runId": "run_xyz789", "messages": [ { "role": "user",  
"content": "What is the stock price of MSFT?" } ] }
```

- `threadId` -- identifies the conversation thread. Reuse across messages to maintain history.
- `runId` -- unique identifier for this specific run.
- `messages` -- array of message objects with role and content.

Response Format

The response is an SSE stream:

```
Content-Type: text/event-stream Cache-Control: no-cache Connection: keep-alive
```

Each event is a JSON object on a `data:` line, with a `type` field indicating the event type.

Event Types

Text Streaming

Text messages are streamed in three phases:

```
data: {"type": "TEXT_MESSAGE_START", "messageId": "msg_001"} data: {"type":  
"TEXT_MESSAGE_CONTENT", "messageId": "msg_001", "delta": "The current"} data: {"type":  
"TEXT_MESSAGE_CONTENT", "messageId": "msg_001", "delta": " stock price of MSFT is $425.30."}  
data: {"type": "TEXT_MESSAGE_END", "messageId": "msg_001"}
```

Tool Execution Lifecycle

Tool calls are bracketed by start and end events:

```
data: {"type": "TOOL_CALL_START", "toolCallId": "tc_001", "toolName": "stock_quote", "args":  
"{\"symbol\": \"MSFT\"}"} data: {"type": "TOOL_CALL_END", "toolCallId": "tc_001", "result":  
"{\"symbol\": \"MSFT\", \"price\": 425.30}"}
```

State Updates

State events push structured data to the frontend state panel. Two behaviors exist:

Snapshot replaces the entire state panel content:

```
data: {"type": "STATE_SNAPSHOT", "snapshot": {"symbol": "MSFT", "price": 425.30, "change": "+1.2%"}}
```

Delta appends to or merges with the existing state panel content:

```
data: {"type": "STATE_DELTA", "delta": {"latestAlert": "Price crossed $425 threshold"}}
```

Agent Step Tracking

Steps track higher-level phases of agent execution:

```
data: {"type": "STEP_STARTED", "stepName": "Analyzing market data"} data: {"type": "STEP_FINISHED", "stepName": "Analyzing market data"}
```

Chain-of-Thought Reasoning

Reasoning messages expose the model's thought process:

```
data: {"type": "REASONING_START"} data: {"type": "REASONING_MESSAGE_CONTENT", "delta": "I need to look up the current price..."} data: {"type": "REASONING_END"}
```

The frontend displays reasoning blocks temporarily and clears them after 5 seconds.

Run Lifecycle

```
data: {"type": "RUN_STARTED", "runId": "run_xyz789"} data: {"type": "RUN_FINISHED", "runId": "run_xyz789", "outcome": "completed"}
```

When outcome is "interrupt", the frontend shows approve/deny buttons for the user to continue or cancel the interrupted action.

Error Reporting

```
data: {"type": "RUN_ERROR", "runId": "run_xyz789", "error": "Rate limit exceeded. Retry after 60 seconds."}
```

Why SSE over WebSockets

The AG-UI protocol uses server-sent events rather than WebSockets for several reasons:

- **Unidirectional fit** -- the agent streams output to the client; the client sends new requests as separate HTTP POSTs. This matches the SSE model naturally.
- **HTTP compatibility** -- SSE runs over standard HTTP, so it works with existing load balancers, reverse proxies, and CDNs without special configuration.
- **Auto-reconnection** -- the browser's EventSource API reconnects automatically on connection loss. The frontend fetch-based SSE reader handles this as well.

- **Simpler infrastructure** -- no WebSocket upgrade handshake, no connection state to manage on the server, no sticky sessions required.
- **Firewall friendly** -- SSE uses standard HTTP ports and protocols, avoiding firewall rules that block WebSocket upgrades.

Cancellation

The backend respects `HttpContext.RequestAborted` as a `CancellationToken`. When the client disconnects (navigates away, closes the tab, or explicitly aborts the fetch), the token fires and the agent stops processing. This prevents wasted compute on abandoned requests.

ToolStateMapping

The `ToolStateMapping` record and `ToolStateBehavior` enum control how tool results appear in the state panel:

```
public enum ToolStateBehavior { Snapshot, // Replaces the entire state panel Delta, // Appends to existing state } public sealed record ToolStateMapping(string ToolName, ToolStateBehavior Behavior);
```

Tool state mappings are registered in each capability's service extensions. When the AG-UI stream encounters a result from a mapped tool, `AGUIStreamOptions.MapResult` routes it through either `ToStateSnapshot` or `ToStateDelta`, which emit the corresponding SSE event.

Registered mappings:

Tool Name	Behavior	Source
`list_workflows`	Snapshot	WorkflowServiceExtensions
`run_workflow`	Delta	WorkflowServiceExtensions
`run_sequential_workflow`	Delta	WorkflowServiceExtensions
`run_concurrent_workflow`	Delta	WorkflowServiceExtensions
`list_scheduled_tasks`	Snapshot	SchedulerServiceExtensions
`stock_quote`	Snapshot	MarketDataServiceExtensions
`stock_history`	Snapshot	MarketDataServiceExtensions
`edgar_company_facts`	Snapshot	MarketDataServiceExtensions
`list_available_agents`	Snapshot	OrchestrationServiceExtensions

Reference: Security and Governance

Governance Kernel

The `GovernanceKernel` (from the `Microsoft.AgentGovernance` NuGet package) is the central governance object. It is constructed from `GovernanceOptions` and exposes three components:

- **PolicyEngine** -- evaluates tool calls against the governance policy rules.
- **AuditEmitter** -- emits structured audit events for policy checks, violations, and blocked calls.
- **InjectionDetector** -- scans system instructions for prompt injection patterns.

Registration

Call `AddAiAgentCanvasSecurity()` in `Program.cs` before any other AI Agent Canvas service registration:

```
builder.Services.AddAiAgentCanvasSecurity(builder.Configuration);
```

This registers the following components as singletons:

Component	Purpose
<code>`GovernanceKernel`</code>	Central governance object
<code>`PolicyEngine`</code>	Policy evaluation (extracted from kernel)
<code>`AuditEmitter`</code>	Audit event emission (extracted from kernel)
<code>`GovernanceContextProvider`</code>	Scans system instructions for prompt injection
<code>`GovernedMcpGateway`</code>	Evaluates tool calls against MCP gateway rules
<code>`GovernanceToolWrapper`</code>	Wraps all <code>`AIFunction`</code> instances with governance checks
Rate limiter	Fixed-window rate limiter (ASP.NET built-in)

On the middleware side, `UseAiAgentCanvasSecurity()` enables the rate limiter, applies security headers, and subscribes to governance audit events for logging.

Tool-Call Governance Pipeline

Every tool call passes through a five-step governance pipeline:

```
1. Tool call received | 2. Build McpGatewayRequest (agentId, toolName, payload) | 3. Evaluate against policy rules via GovernedMcpGateway | 4. Decision: Allow / Block / Audit | 5. Execute the tool (if allowed) or return a JSON error (if blocked)
```

GovernedAIFunction

`GovernedAIFunction` extends `DelegatingAIFunction` (from `Microsoft.Extensions.AI`) and intercepts every tool call. It:

- Serializes the call arguments to JSON.
- Calls `GovernedMcpGateway.Evaluate()` to get an allow/block decision.
- Emits an audit event via `AuditEmitter` (either `PolicyCheck` or `ToolCallBlocked`).
- If blocked, returns a JSON error string: `{"error": "Tool 'X' was blocked by governance policy.", "status": "..."}.`
- If allowed, delegates to the inner function via `base.InvokeCoreAsync()`.

GovernanceToolWrapper

`GovernanceToolWrapper` implements `IToolGovernanceWrapper`. At startup, the orchestration layer calls `Wrap()` on every registered `AIFunction`, replacing the original with a `GovernedAIFunction`. This ensures governance applies to all tools without requiring each tool to implement its own checks.

Policy Format

Governance policies are defined in YAML. The policy file path is set via `Security:PolicyPath` in configuration (default: `governance-policy.yaml`).

Structure

```
name: AiAgentCanvas-default description: Default governance policy for AiAgentCanvas rules: -
name: block-dangerous-tools scope: tool_call action: deny condition: tool_name: in:
[run_script] reason: "Shell execution requires explicit approval" - name:
restrict-file-write-paths scope: tool_call action: deny condition: tool_name: equals:
write_file path: matches: "^(/etc|/var|C:\\\\Windows|C:\\\\Program Files)" reason: "Writing to
system directories is blocked" - name: block-private-mcp-endpoints scope: tool_call action:
deny condition: tool_name: equals: connect_mcp_server endpoint: matches: "(localhost|127\\.0\\.
0\\.1|169\\.254\\.169\\.254|10\\.172\\.1[6-9]|2[0-9]|3[01])\\.192\\.168\\.)" reason: "MCP
connections to private/internal addresses are blocked (SSRF protection)" - name:
allow-all-other scope: tool_call action: allow condition: {}
```

Rule Fields

Field	Description
<code>`name`</code>	Unique identifier for the rule
<code>`scope`</code>	What the rule applies to (currently <code>`tool_call`</code>)
<code>`action`</code>	<code>`allow`</code> , <code>`deny`</code> , or <code>`audit`</code>
<code>`condition`</code>	Match criteria using operators (see below)
<code>`reason`</code>	Human-readable explanation shown when the rule fires

Condition Operators

Operator	Description	Example
<code>`equals`</code>	Exact string match	<code>`tool_name: { equals: write_file }`</code>
<code>`in`</code>	Matches any value in a list	<code>`tool_name: { in: [run_script, exec] }`</code>
<code>`matches`</code>	Regular expression match	<code>`path: { matches: "^/etc" }`</code>

Conflict Strategy

When multiple rules match a tool call, `ConflictResolutionStrategy.DenyOverrides` applies: any matching deny rule wins regardless of allow rules. This is the default and recommended setting.

Rate Limiting

Rate limiting uses ASP.NET's built-in fixed-window rate limiter with a policy named "agent".

Parameter	Value
Window	1 minute
Permit limit	Configurable via <code>`Security:RateLimitPerMinute`</code> (default: 30)
Queue limit	0 (no queuing; excess requests are rejected immediately)

When the limit is exceeded, the server returns HTTP 429 with a JSON body:

```
{ "error": "Rate limit exceeded. Try again later.", "retryAfterSeconds": 60 }
```

Security Headers

Three security headers are applied to every HTTP response via inline middleware in `UseAiAgentCanvasSecurity()`:

Header	Value	Purpose
<code>`X-Content-Type-Options`</code>	<code>`nosniff`</code>	Prevents MIME-type sniffing
<code>`X-Frame-Options`</code>	<code>`DENY`</code>	Prevents the page from being embedded in frames
<code>`Referrer-Policy`</code>	<code>`strict-origin-when-cross-origin`</code>	Limits referrer information sent to other origins

Production Checklist

Before deploying to production, verify each item:

- **Set real API keys** -- replace placeholder values in `AIFoundry:Endpoint` and `AIFoundry:Key` with production credentials, or enable `UseAzureCredential` for managed identity.
- **Configure governance policies** -- review and customize `governance-policy.yaml` for your security requirements. Add deny rules for any tools that should not run in production.
- **Enable rate limiting** -- set `Security:RateLimitPerMinute` to an appropriate value for your expected load.
- **Review tool allowlists** -- audit the system tools configuration (`AllowedCommands`) and remove any commands not needed in production.
- **Configure HTTPS** -- ensure the application runs behind HTTPS termination (reverse proxy or Kestrel HTTPS configuration).
- **Set logging level** -- change `Logging:LogLevel:Default` to `Warning` or `Error` to reduce log volume and avoid logging sensitive data.
- **Remove DevUI in production** -- the `AddDevUI()` and `MapDevUI()` calls should be wrapped in an environment check so the development UI is not exposed in production.
- **Review MCP connection security** -- the default policy blocks MCP connections to private/internal addresses. Verify this rule is active and add additional endpoint restrictions as needed.

Reference: Platform Internals

Context Provider Chain

Context providers extend `AIContextProvider` (from `Microsoft.Agents.AI`) and run in DI registration order. Each provider's `ProvideAIContextAsync` method appends content to `AIContext.Instructions`, building up the system prompt that the agent receives.

Order	Provider	Source	What It Injects
1	<code>`GovernanceContextProvider`</code>	<code>AiAgentCanvas.Security</code>	Scans existing instructions for prompt injection; emits audit events if detected (does not modify instructions)
2	<code>`PersonaContextProvider`</code>	<code>AiAgentCanvas.AgentData.Personas</code>	Active persona's instructions, or the default system prompt if no persona is active
3	<code>`PersistentContextProvider`</code>	<code>AiAgentCanvas.AgentData.Context</code>	All saved context entries (facts, notes, preferences)
4	<code>`EntityContextProvider`</code>	<code>AiAgentCanvas.AgentData.Entities</code>	Entity index listing all known entities and their types
5	<code>`UserProfileContextProvider`</code>	<code>AiAgentCanvas.AgentData.Profiles</code>	Active user profile context (role, timezone, preferences)
6	<code>`GuardrailContextProvider`</code>	<code>AiAgentCanvas.AgentData.Guardrails</code>	All enabled guardrail rules
7	<code>`DynamicToolContextProvider`</code>	<code>AiAgentCanvas.Orchestration.Skills</code>	Injects dynamically registered tools into <code>`AIContext.Tools`</code> (the only provider that modifies tools rather than instructions)
8	<code>`RagContextProvider`</code>	<code>AiAgentCanvas.Capabilities.Rag</code>	Retrieves relevant document chunks via hybrid search and appends them as numbered citations (only active when RAG is configured)

The registration order follows the order of `AddAiAgentCanvas*()` calls in `Program.cs`: Security first, then Personas, Context, Entities, Profiles, Guardrails, Skills, and finally RAG.

MarkdownFile Utility

`MarkdownFile` (`AiAgentCanvas.Abstractions` namespace) is a utility for reading and writing markdown files with YAML frontmatter. All agent data domains use it for persistence.

Methods

Method	Signature	Description
<code>`Parse`</code>	<code>`static MarkdownFile? Parse(string filePath)`</code>	Reads a file from disk and parses its frontmatter and body. Returns <code>`null`</code> if the file does not contain valid frontmatter delimiters (<code>`---`</code>).
<code>`ParseContent`</code>	<code>`static MarkdownFile? ParseContent(string content, string filePath)`</code>	Parses frontmatter and body from a string. Returns <code>`null`</code> if the content does not start with <code>`---`</code> or lacks a closing delimiter.
<code>`Write`</code>	<code>`static void Write(string filePath, Dictionary<string, string> frontmatter, string body)`</code>	Creates the parent directory if needed, serializes frontmatter as <code>`key: value`</code> lines between <code>`---`</code> delimiters, appends the body, and writes UTF-8.
<code>`LoadAll`</code>	<code>`static List<MarkdownFile> LoadAll(string directory, string pattern = "*.md")`</code>	Returns all parseable markdown files from a directory.
<code>`SanitizeFileName`</code>	<code>`static string SanitizeFileName(string name)`</code>	Lowercases the name and replaces spaces and underscores with hyphens.
<code>`Get`</code>	<code>`string Get(string key, string fallback = "")`</code>	Looks up a frontmatter key; returns <code>`fallback`</code> if missing.
<code>`GetBool`</code>	<code>`bool GetBool(string key, bool fallback = false)`</code>	Looks up a frontmatter key and parses it as a boolean.

Properties

Property	Type	Description
<code>`Frontmatter`</code>	<code>`Dictionary<string, string>`</code>	Parsed key-value pairs from the YAML frontmatter block

Property	Type	Description
<code>`Body`</code>	<code>`string`</code>	Everything after the closing <code>`---`</code> delimiter
<code>`FilePath`</code>	<code>`string`</code>	The file path this instance was parsed from

Tool Design Guidelines

Follow these rules when building tools for AI Agent Canvas:

- **Use descriptive names matching the domain.** Tool names should read naturally: `stock_quote`, `list_personas`, `run_workflow`. Avoid generic names like `execute` or `process`.
- **Write clear [Description] attributes.** The description is included in the model's prompt. Be specific about what the tool does, what it returns, and when to use it.
- **Return structured JSON.** Tools should return JSON-serializable objects. The AG-UI protocol and state panel depend on structured output.
- **Handle errors gracefully.** Return error information as part of the result rather than throwing exceptions. The model can interpret and relay error messages to the user.
- **Keep tools focused.** Each tool should do one job. Prefer two small tools over one tool with a `mode` parameter.
- **Accept CancellationToken.** Pass the token through to all async operations. This allows the system to cancel work when the user disconnects.
- **Document parameters with [Description].** Every parameter should have a description attribute so the model knows what values to pass.

Seed Interface Reference

Seeds provide default data for each agent domain. They are resolved from DI at startup. If the corresponding file does not already exist on disk, the seed data is persisted. Seeds never overwrite files that a user has manually edited.

Interface	Concrete Type	Properties	Persistence Path
<code>`IPersonaSeed`</code>	<code>`PersonaSeed`</code>	<code>`Name`</code> , <code>`Description`</code> , <code>`Instructions`</code>	<code>./agent-data/orchestrator/agent/personas/`</code>
<code>`IContextSeed`</code>	<code>`ContextSeed`</code>	<code>`Topic`</code> , <code>`Type`</code> , <code>`Tags`</code> , <code>`Content`</code>	<code>./agent-data/orchestrator/agent/context/`</code>
<code>`IWorkflowSeed`</code>	<code>`WorkflowSeed`</code>	<code>`Name`</code> , <code>`Description`</code> , <code>`Tags`</code> , <code>`Content`</code>	<code>./agent-data/orchestrator/agent/workflows/`</code>
<code>`IEntitySeed`</code>	<code>`EntitySeed`</code>	<code>`Name`</code> , <code>`Type`</code> , <code>`Tags`</code> , <code>`Content`</code>	<code>./agent-data/orchestrator/agent/entities/`</code>

Interface	Concrete Type	Properties	Persistence Path
<code>IGuardrailSeed`</code>	<code>GuardrailSeed`</code>	<code>`Name`</code> , <code>`Severity`</code> , <code>`Enabled`</code> , <code>`Rule`</code>	<code>./agent-data/orchestrator/agent/guardrails/`</code>
<code>ISkillSeed`</code>	<code>SkillSeed`</code>	<code>`Name`</code> , <code>`Description`</code> , <code>`PromptTemplate`</code>	(via SkillRegistry)
<code>IUserProfileSeed`</code>	<code>UserProfileSeed`</code>	<code>`Name`</code> , <code>`Role`</code> , <code>`Timezone`</code> , <code>`Content`</code>	<code>./agent-data/orchestrator/agent/profiles/`</code>
<code>IMcpConnectionSeed`</code>	<code>McpConnectionSeed`</code>	<code>`Name`</code> , <code>`Endpoint`</code> , <code>`Transport`</code>	(via MCP connection manager)
<code>IAgentToolsSeed`</code>	<code>AgentToolsSeed`</code>	<code>`AgentName`</code> , <code>`ToolNames`</code> (list)	(in-memory tool assignment)
<code>IGoalSeed`</code>	<code>GoalSeed`</code>	<code>`Name`</code> , <code>`Description`</code> , <code>`Priority`</code> , <code>`AcceptanceCriteria`</code> , <code>`AssignedAgent`</code> , <code>`Content`</code>	<code>./agent-data/orchestrator/agent/goals/`</code>

Each domain also supports user-created data that persists under the `user/` subtree (e.g., `./agent-data/orchestrator/user/personas/`) and shared data under `./agent-data/shared/`.

Seed Behavior

- At startup, the DI container resolves all registered `ISeed` implementations for each domain.
- For each seed, the store checks if a file with the sanitized name already exists.
- If the file does not exist, the seed data is written using `MarkdownFile.Write()`.
- If the file exists, the seed is skipped -- manual edits are preserved.

RAG Pipeline Internals

The RAG (Retrieval-Augmented Generation) pipeline is conditionally enabled when `AIFoundry:EmbeddingDeploymentName` is configured. It adds relevant document context to the agent's system prompt before each response.

DocumentChunker

The `DocumentChunker` class splits text into overlapping chunks for ingestion.

Parameter	Default	Description
<code>`ChunkSize`</code>	512	Maximum characters per chunk
<code>`ChunkOverlap`</code>	64	Characters of overlap carried from the previous chunk

The chunking algorithm:

- Split text by double-newline (paragraph boundaries).
- Accumulate paragraphs into a buffer until `ChunkSize` would be exceeded.
- Flush the buffer as a `DocumentChunk`, carrying the last `ChunkOverlap` characters forward.
- If a single paragraph exceeds `ChunkSize`, split it further by sentence endings (. , ! , ?).
- Discard chunks shorter than 20 characters.

Hybrid Search

Hybrid search combines vector similarity with keyword matching. The weights are configurable via `RagSearchOptions`:

Component	Weight	Method
Vector search	0.7 (default)	Cosine similarity against stored embeddings
Keyword search	0.3 (default)	SQLite FTS5 BM25 ranking

The `IHybridSearchable` interface:

```
public interface IHybridSearchable { IAsyncEnumerable<(DocumentRecord Record, double? Score)>  
  HybridSearchAsync( ReadOnlyMemory<float> queryEmbedding, int top, RagSearchOptions? options =  
    null, CancellationToken ct = default); }
```

`RagSearchOptions` fields: `SourceFilter`, `TagFilter`, `KeywordQuery`, `KeywordWeight` (default 0.3f), `VectorWeight` (default 0.7f).

The final score for each document is: $(\text{VectorWeight} * \text{vectorScore}) + (\text{KeywordWeight} * \text{keywordScore})$.

FTS5 Schema

The SQLite FTS5 virtual table is created alongside the main documents table:

```
CREATE VIRTUAL TABLE IF NOT EXISTS [{collection}_fts] USING fts5(id UNINDEXED, text,  
content=[{collection}], content_rowid=rowid)
```

- `id` is stored but not indexed (marked `UNINDEXED`).
- `text` is the searchable column.
- The FTS table is a content-sync table that mirrors the main documents table.

Keyword scores are normalized as $1.0 / (1.0 + |\text{rank}|)$ where `rank` is the FTS5 BM25 score.

LLM Reranking

After hybrid search retrieves the top candidates, an LLM reranker narrows the results:

- Retrieve `retrieveK` candidates (default: 10) via hybrid search.
- Truncate each candidate's text to 300 characters.

- Send a prompt to the LLM asking it to return a JSON array of chunk indices ranked by relevance (e.g., [2, 0, 4, 1, 3]).
- Parse the JSON array, deduplicate indices, and take the top topK (default: 3).
- On any parse or LLM error, fall back to the original ordering truncated to topK.

LLM reranking parameters: Temperature = 0, MaxOutputTokens = 100.

DocumentRecord

```
public sealed class DocumentRecord { [VectorStoreKey] public string Id { get; set; }
[VectorStoreData] public string Text { get; set; } [VectorStoreData] public string? Source {
get; set; } [VectorStoreData] public string? Tags { get; set; } [VectorStoreData] public
string? MetadataJson { get; set; } [VectorStoreVector(1536, DistanceFunction =
CosineSimilarity)] public ReadOnlyMemory<float> Embedding { get; set; } }
```

The embedding dimension is 1536 (compatible with OpenAI text-embedding-ada-002 and similar models). The VectorStoreVector attribute specifies cosine similarity as the distance function.